GOA UNIVERSITY

Master of Computer Applications (MCA)

CSA-5005

Object Oriented Technology Lab

Mini Project Report

QuizClass

A Console-Based Quiz System in Java

Class-code joining, enforced timer, auto-submit and instant grading

Submitted by

Name	Roll No.
Aayush Mahale	2601
Ajay Naik	2606

Under the guidance of

Hanumant Redkar
Sarvesh Velip

26 October 2026

Acknowledgement

We would like to thank our faculty, **Hanumant Redkar** and **Sarvesh Velip**, for their guidance and for the clear expectations they set for the Object Oriented Technology laboratory.

We are grateful to Goa University for the laboratory facilities and for the opportunity to build a complete project as part of this course.

The two of us built QuizClass as a team. We thank each other for the steady cooperation: for agreeing the design decisions early, for reviewing each other's code and for fixing integration problems quickly when the two halves of the program were joined.

Aayush Mahale (2601)
Ajay Naik (2606)

Abstract

Checking whether a class has understood a topic is slow when quizzes are collected and marked by hand. QuizClass is a console-based quiz system written in pure Java that removes the marking. A teacher creates a quiz and receives a four-digit class code; students join by typing the code, answer multiple-choice questions against a countdown, and are graded instantly. When time runs out, the quiz is submitted automatically with the answers given so far, and a student cannot submit the same quiz twice, even after a restart.

The design is deliberately object oriented. An abstract `Question` with a concrete `MCQ` subclass shows abstraction and inheritance; `Quiz` composes a `Timer` and its questions; `Student`, `StudentResponse`, `Teacher` and `QuizManager` divide the remaining responsibilities; and a watcher thread with synchronised methods implements the auto-submit safely. Quizzes and student responses are stored in plain text files and reloaded at start-up.

This report describes the requirements, the brainstorming that led to the project, a comparison with existing tools, the class, use case, sequence and state diagrams, the implementation, the file formats, and the testing with screenshots of a complete session. It also states the known limitations honestly and suggests future work.

Keywords: Java, object-oriented design, quiz system, timer, auto-submit, multithreading, file persistence.

Table of Contents

1. Introduction 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope 2
1.5 Tools and Technologies 2
1.6 Team and Division of Work 2
1.7 Organisation of the Report 2
2. Requirements and Scope 3
2.1 Users of the System 3
2.2 Functional Requirements 3
2.3 Non-Functional Requirements 3
2.4 Scope Boundaries 4
2.5 Design Decisions Fixed Early 4
2.6 Assumptions 5
3. Brainstorming: Ideas Considered 6
3.1 How the Project Idea Was Chosen 6
3.2 Project Ideas 6
3.3 Technical Alternatives 7
3.4 How the Design Evolved 7
4. Comparison with Existing Tools 9
4.1 Purpose of the Comparison 9
4.2 Tools Compared 9
4.3 Findings 9
4.4 What QuizClass Lacks 10
5. System Design 112
5.1 Architecture Overview 112
5.2 Class Diagram 112
5.3 Responsibilities of the Classes 13
5.4 Object-Oriented Concepts Used 143
5.5 Use Case Diagram 154
5.6 Sequence Diagrams 166
5.7 Life Cycle of a Quiz 187
5.8 Threading Design 198
6. Implementation 19

6.1 Project Structure and How to Run 19
6.2 Question and MCQ 19
6.3 Timer 20
6.4 Quiz 21
6.5 StudentResponse 22
6.6 Student 23
6.7 Teacher and QuizManager 25
6.8 Demo: the Console Driver 26
6.9 StudentApp and Main 27
7. Persistence and File Formats 28
7.1 Overview 28
7.2 The Quiz File 28
7.3 The Response File 28
7.4 Loading at Start-up 29
7.5 Duplicate Prevention Across Restarts 30
7.6 Error Handling 30
7.7 What Is Not Persisted 30
8. Testing and Results 31
8.1 Test Approach 31
8.2 Test Cases 31
8.3 Walk-through of a Complete Session 32
8.4 Summary of Results 39
9. Limitations and Future Work 40
9.1 Known Limitations 40
9.2 Future Work 41
10. Conclusion 42
References 43

1. Introduction

1.1 Background

A short quiz is one of the quickest ways for a teacher to find out whether a class has understood a topic. On paper, however, the quiz has to be collected, marked and returned, and that takes far longer than the quiz itself. Online tools remove the marking, but most of them are large platforms that need accounts, a browser and a network connection, and their internals are hidden from a student who wants to understand how such a system is actually built.

QuizClass is a small quiz system written in pure Java for the Object Oriented Technology laboratory (CSA-5005). A teacher creates a quiz and receives a four-digit class code. Students join the quiz by typing that code, answer the questions against a countdown, and are graded instantly. When the time runs out the quiz is submitted automatically with whatever has been answered, so no student can keep working past the deadline. The whole system runs in a console, uses only the Java standard library and stores its data in plain text files.

1.2 Problem Statement

Teachers need to check student understanding quickly, without collecting papers and without marking by hand. A solution has to let the teacher prepare a quiz once, let students take it under a fair and enforced time limit, grade the answers without human effort, and show the results straight away. Because this is a laboratory project, the solution must also make its object-oriented design clearly visible: the classes, their responsibilities and the relationships between them should be easy to read in the code and to explain in a demonstration.

1.3 Objectives

1. Design a set of cooperating Java classes that model questions, quizzes, teachers, students and student responses.
2. Let a teacher create a quiz and obtain a unique four-digit class code, and let students join the quiz using only that code.
3. Enforce a time limit: answers are refused after time-up, and the quiz is auto-submitted when the timer ends.
4. Grade responses automatically and instantly, and show each student a percentage.
5. Let the teacher view live responses, the results of every student and the class average.
6. Persist quizzes and responses in text files so that they survive a restart, and refuse a second attempt by the same student.
7. Demonstrate abstraction, inheritance, polymorphism, encapsulation and composition clearly, together with basic multithreading for the auto-submit.

1.4 Scope

The system covers quiz creation, code-based joining, a timer with auto-submit, automatic grading and the display of results. The following are deliberately outside the scope of this project: a database, a graphical user interface, real networking between separate machines, analytics and a mobile application. These exclusions keep the project small enough to finish and test properly in four weeks, and they are revisited as future work in Chapter 9.

1.5 Tools and Technologies

- **Language:** Java, using only the standard library: `java.util` collections, `java.nio.file` for file input and output, and `java.util.concurrent.atomic` for a flag shared between threads.
- **Interface:** a text console driven by menus; no graphical toolkit.
- **Storage:** plain text files with a pipe-separated format, one file per quiz and one per student response.
- **Version control:** Git, with each team member working on a separate branch and merging through reviewed changes.
- **Diagrams:** class, use case, sequence and state diagrams, included in Chapter 5.

1.6 Team and Division of Work

The project was built by a team of two. The work was divided along the natural boundary of the design: one member built the quiz-side classes and the console driver, the other built the student-side classes.

Table 1.1: Division of work

Member	Roll No.	Classes and responsibilities
Ajay Naik	2606	Question, Quiz, Teacher, QuizManager, Timer and the Demo console driver; file persistence for quizzes; overall architecture
Aayush Mahale	2601	MCQ, Student, StudentResponse and the StudentApp prototype; answer validation; file persistence for student responses

Integration was done together: the two halves were merged, run against each other, and corrected until a complete session (create, start, join, answer, submit, grade, view results) worked from the console.

1.7 Organisation of the Report

Chapter 2 states the requirements and the design decisions that were fixed early. Chapter 3 records the brainstorming: the ideas that were considered, which were accepted and which were rejected. Chapter 4 compares QuizClass with existing tools. Chapter 5 presents the design with the class, use case, sequence and state diagrams. Chapter 6 walks through the implementation class by class, and Chapter 7 describes the file formats. Chapter 8 documents the testing with

screenshots of the working product. Chapter 9 lists the known limitations and possible future work, and Chapter 10 concludes.

2. Requirements and Scope

2.1 Users of the System

There are two kinds of users. A **teacher** creates and manages quizzes: they write the questions, start and end the quiz and read the responses and results. A **student** joins a quiz with its class code, answers the questions and sees the score. A third, non-human actor appears in the design: the **timer**, which closes the quiz when its time is up and so triggers the automatic submission.

2.2 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR-1	The teacher can create a quiz with a title, a duration in minutes and a list of questions. The system issues a unique four-digit class code.
FR-2	Each question is a multiple-choice question with at least two options. The correct answer is stored as a single option letter.
FR-3	A quiz is saved to a file when it is created, and all saved quizzes are loaded when the program starts.
FR-4	The teacher starts a quiz, which starts its countdown. A quiz that has ended, or whose time has run out, cannot be started again.
FR-5	A student joins by entering the class code, a student ID and a name. An unknown class code is rejected.
FR-6	A student answers each question with a single letter. An invalid letter is rejected and the question is asked again.
FR-7	Answers are refused once the quiz is closed. When the time runs out, the quiz is submitted automatically with the answers given so far.
FR-8	Grading is automatic and immediate. The score is the number of correct answers as a percentage of all questions; unanswered questions count as wrong.
FR-9	A student cannot submit the same quiz twice, including after the program has been restarted. A student who returns sees the saved score.
FR-10	The teacher can view live responses (who joined and how many questions each has answered), the results of all students and the class average.
FR-11	The teacher can end a quiz early; ending grades any response that is still unsubmitted.
FR-12	Student responses are saved to files and are visible to the teacher after a restart.

2.3 Non-Functional Requirements

- **Simplicity and readability:** each class has one clear responsibility, and method names describe what they do, because the design has to be explained in a viva.
- **Portability:** pure Java with no external libraries, so the project compiles and runs on any machine with a Java Development Kit.

- **Robustness:** invalid input never crashes the program; a damaged quiz file is skipped with a warning instead of stopping the start-up.
- **Correctness under concurrency:** the auto-submit thread and the main thread must never both finish the same response.
- **Testability:** every requirement above can be exercised from the console and checked against the saved files.

2.4 Scope Boundaries

Table 2.2: In scope and out of scope

In scope	Out of scope
Quiz creation by a teacher	A database
Code-based joining	A graphical user interface
Timer with auto-submit	Real networking between machines
Automatic grading	Analytics and reports
Display of results	A mobile application

2.5 Design Decisions Fixed Early

Several rules were agreed before coding started, because they affect more than one class. Writing them down avoided integration bugs between the two halves of the team.

Table 2.3: Design decisions

Decision	Reason
A quiz is identified by its **class code**, never by a numeric quiz ID.	The class code is the only value a student ever types, so every lookup (manager, student, file name) uses it. This removes any mismatch between two identifiers.
Answers are stored in a map keyed by **question ID**, never by list position.	The answer key and the student's answers then line up even if the order of questions changes.
An answer is a single letter, validated by `Question.validate`.	Validation belongs to the question type, so the student classes need no knowledge of how a question checks its answers.
Unanswered questions count as wrong.	The score is always correct answers divided by the total number of questions, so skipping a question cannot raise the percentage.
A quiz cannot be restarted after it ends.	A second run would give some students extra time and make results incomparable.
Answering or submitting after time-up is refused, and auto-submit gives no extension.	The deadline is the point of the system; a grace period would defeat it.
A duplicate submission is refused, including across restarts.	The saved response file is checked on join, so closing and reopening the program does not give a second attempt.

2.6 Assumptions

- Teacher and students share one console program and one machine, so students take the quiz one after another in the same session.
- The teacher starts the quiz from the teacher menu before students join.
- Class codes are announced to students by the teacher in the classroom; the system does not distribute them.

3. Brainstorming: Ideas Considered

3.1 How the Project Idea Was Chosen

The assignment asked for a mini-project that shows object-oriented design, to be finished in four weeks by two people. Instead of taking the first idea, we listed several candidate projects and judged each against the same five questions.

1. **OOP depth:** does the problem naturally need abstraction, inheritance, polymorphism, encapsulation and composition, or would the classes be empty containers?
2. **Size:** can two people design, build and test it properly in four weeks?
3. **Demonstration:** can the working system be shown in about five minutes from a console?
4. **Testability:** can every feature be checked with a clear expected result?
5. **Dependencies:** can it be written in pure Java without installing extra libraries?

3.2 Project Ideas

Table 3.1: Candidate projects and decisions

Idea	What it would be	Decision	Main reason
Personal finance tracker	Record income and expenses, categorise them, show monthly totals.	Rejected	Mostly record keeping and arithmetic; the class hierarchy would be shallow and there is little behaviour that differs between object types.
Dungeon crawler	A console game with rooms, monsters, items and combat.	Rejected	A strong fit for inheritance, but the amount of game content and balancing is open-ended, and random events make results hard to test and to demonstrate in five minutes.
Grade management with scholarship and trend analysis	Store marks, compute eligibility for scholarships and analyse trends over semesters.	Rejected	The interesting part would be the statistics, not the object design; it also needs a lot of sample data to look convincing.
Full classroom app with chat and notes	A classroom suite: quizzes, a chat between teacher and students, shared notes.	Rejected	Far too large for four weeks, and chat needs real networking, which was already ruled out.
QuizClass	A teacher creates a quiz; students join with a code, answer under a timer and are graded instantly.	**Accepted**	Small and complete: a clear real-world problem, a natural class hierarchy, a visible concurrency aspect (the timer) and a five-minute demonstration with precise expected outputs.

QuizClass satisfied all five questions at once. In particular, the timer with auto-submit gave the project one feature that goes beyond plain classes and methods, while still being small enough to test exactly.

3.3 Technical Alternatives

A quiz system needs somewhere to keep quizzes and responses, and a way for the teacher side and the student side to see the same data. Several approaches were discussed.

Table 3.2: Storage and communication approaches

Approach	Decision	Reason
HTTP server (teacher and students as clients)	Rejected	Would need request handling, routing and a client, which is a second project on its own and moves the focus away from the class design.
SQLite database	Rejected	Needs an external driver, which breaks the pure-Java rule, and database access code would take time from the object design. A database is also outside the agreed scope.
Sockets between separate programs	Rejected	Too long for the timeline: connection handling, protocol design and failure cases would not fit in four weeks.
Plain text files, one program	**Accepted**	Needs only the standard library, is easy to inspect in a text editor (useful for testing and for the screenshots), and keeps both sides on the same Quiz objects through the QuizManager.

3.4 How the Design Evolved

The class design drawn in the first week was not the final one. Writing and integrating the code showed which parts of the first design were unnecessary or inconsistent, and they were corrected. The table lists the main changes.

Table 3.3: Changes from the Week 1 design to the final code

Week 1 design	Final code	Why it changed
Quizzes found by `quizId`; `StudentResponse` stored a `quizId`.	Quizzes are found only by `classCode`; a response stores the `classCode`.	The class code is the only identifier a student types; two identifiers invited mismatches.
`Student` and `Teacher` had an `email` field; `Student.responses` was a list.	`Student` keeps a map of responses keyed by class code; the student has no email.	A map gives direct lookup by class code; email was never used.
`StudentResponse` held `correctCount`, `totalQuestions` and `getScore()`.	Only `percentage` is kept; it is computed by `calculateScore`.	One number is enough for every use, so the others were removed.
`MultipleChoiceQuestion`.	`MCQ`.	Shorter name; behaviour unchanged.
`Quiz.getQuizByCode`, `toJSON()` and `toString()`.	Lookup is done by `QuizManager`; the quiz is saved in a pipe-separated text format.	Lookup belongs to the manager; the project uses its own simple file format.

Week 1 design	Final code	Why it changed
No notion of a quiz being open or closed.	`Quiz.isOpen()`, `hasEnded`, `Student.autoSubmit` and `registerResponse` were added.	Needed for the timer, the auto-submit and for the teacher to see responses.
Persistence planned for quizzes and responses.	Implemented with `saveToFile` and `loadFromFile` in `Quiz` and `StudentResponse`.	As planned for Week 3.

4. Comparison with Existing Tools

4.1 Purpose of the Comparison

Quizzes with join codes and timers are not a new idea, and it would be wrong to present QuizClass as a replacement for mature products. The aim of this chapter is to place the project honestly: to see what the established tools already do, and to say clearly what QuizClass does differently and what it lacks. The information below comes from the vendors' and universities' published guides [1]–[6], read in October 2026.

4.2 Tools Compared

Table 4.1: Comparison of QuizClass with existing quiz tools

Tool	How people join	Time limit	Grading	Needs
Kahoot	A game PIN that the host's session generates; players enter it at kahoot.it or in the app [5].	A timer per question, paced by the host.	Automatic; scores and a leaderboard.	Internet; a host running a live game.
Mentimeter	A join code at menti.com, or a QR code or link [3], [4].	Quiz slides; a countdown can be set for a question.	Automatic for quiz slides.	Internet; a presenter.
Google Forms (quiz mode)	A link to the form.	No built-in timer or auto-submit; third-party add-ons provide them [6].	Automatic in quiz mode.	Internet; a Google account for the teacher.
Moodle	Enrolled users log in and open the quiz.	A time limit per attempt; open attempts are submitted automatically by default [1], [2].	Automatic, with a gradebook.	A server, user accounts and an administrator.
QuizClass	A four-digit class code typed by the student.	One time limit for the whole quiz, enforced; auto-submit at time-up.	Automatic and instant; percentage.	Only a Java runtime and a console.

4.3 Findings

The comparison shows that the individual features of QuizClass are not unique. Code-based joining is the normal way to enter a live quiz in Kahoot and Mentimeter, and an enforced time limit with automatic submission is a standard option in Moodle. What can honestly be said is narrower:

- QuizClass combines a join code, an enforced whole-quiz time limit, automatic submission and instant grading in one very small system that needs **no accounts, no browser and no network**.
- Its code is short enough to be read and explained completely, which is the purpose of a laboratory project. The commercial tools are black boxes.
- Google Forms has no timer of its own, so the combination of join code, time limit and auto-submit has to be assembled from add-ons there [6].

4.4 What QuizClass Lacks

- It runs on one machine in one console; students cannot join from their own devices.
- It has one question type (multiple choice), no images, no question banks and no randomisation.
- It has no accounts, so a student is identified only by the ID that they type.
- It has no gradebook, analytics or export, only a per-quiz result list with a class average.

These are the trade-offs of keeping the project small. They are discussed again as limitations and future work in Chapter 9.

5. System Design

5.1 Architecture Overview

QuizClass is organised in three groups of classes. The **domain model** holds the data and rules: `Question` and its subclass `MCQ`, `Quiz`, `Timer` and `StudentResponse`. The **actors** act on the model: `Teacher` creates quizzes and reads results, and `Student` joins, answers and submits. `QuizManager` is the single registry through which both actors find the same `Quiz` object by class code. Finally, `Demo` is the console driver: it shows the menus, reads the input and calls the other classes, and contains no grading logic.

Figure 5.1 shows the classes and the relationships between them. The full diagram, with every attribute and method, is given in Figure 5.2.

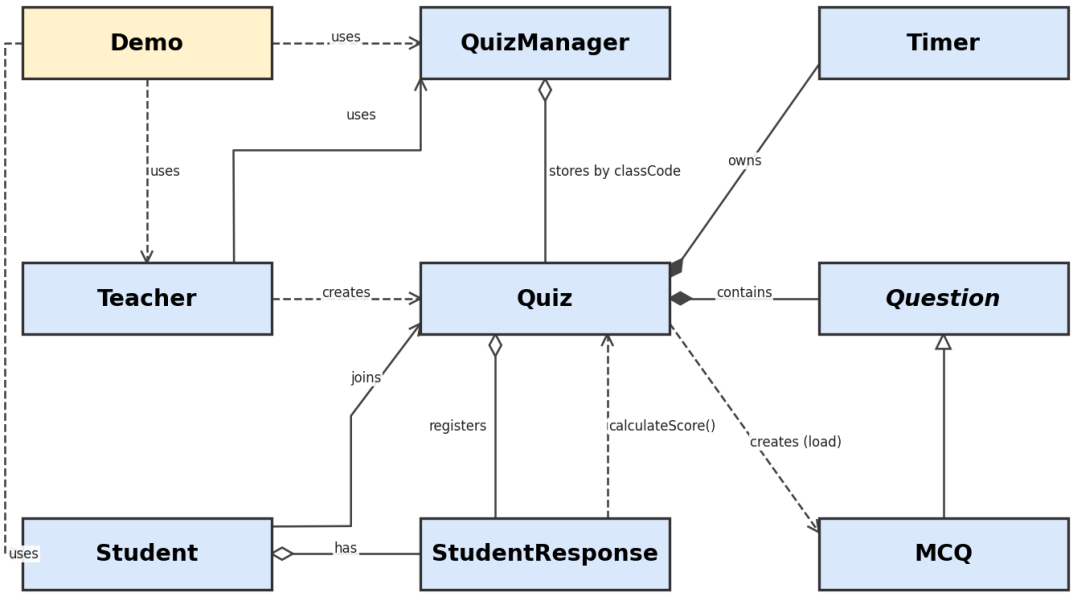


Figure 5.1: Class overview and relationships of QuizClass

5.2 Class Diagram

The detailed class diagram follows on a landscape page. The notation is standard UML: a filled diamond marks composition (the whole creates and owns the part), a hollow diamond marks aggregation (the whole refers to parts that exist elsewhere), a dashed arrow marks a dependency, and a hollow triangle marks inheritance. The abstract class `Question` and its abstract methods are written in italics.

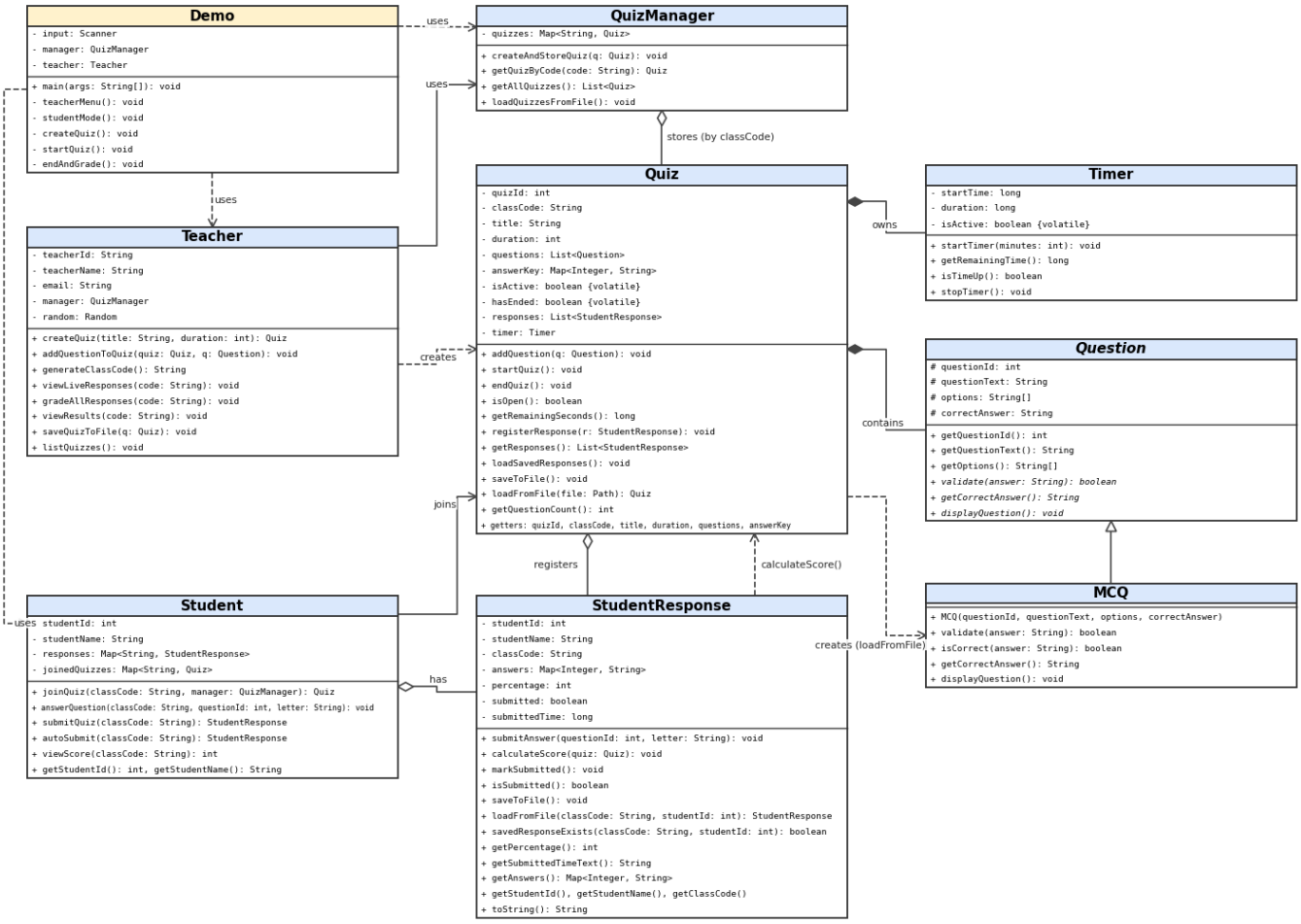


Figure 5.2: Detailed class diagram of QuizClass

5.3 Responsibilities of the Classes

Table 5.1: Classes and their responsibilities

Class	Kind	Responsibility	Works with
Question	Abstract class	Holds the data every question shares (ID, text, options, correct answer) and declares `validate`, `getCorrectAnswer` and `displayQuestion`.	Quiz, Student
MCQ	Concrete class	A multiple-choice question; options are lettered A, B, C…; a valid answer is one existing letter.	Question (parent)
Quiz	Concrete class	Holds the questions, the answer key, the timer and the registered responses; knows whether it is open; saves and loads itself.	Timer, Question, StudentResponse
Timer	Utility class	A passive countdown: starts, reports the remaining time, says whether time is up, can be stopped.	Quiz
StudentResponse	Concrete class	One student's answers to one quiz, keyed by question ID; calculates the percentage; saves and loads itself.	Quiz, Student
Student	Concrete class	Joins a quiz by class code, answers, submits, auto-submits and reports the score.	QuizManager, Quiz, StudentResponse
Teacher	Concrete class	Creates quizzes with a unique class code, adds questions, shows live responses and results, grades unsubmitted responses.	QuizManager, Quiz
QuizManager	Utility class	Stores every quiz by class code and loads saved quizzes at start-up.	Quiz, Teacher, Student
Demo	Driver class	Console menus for teacher mode and student mode; runs the watcher thread for auto-submit.	All of the above

5.4 Object-Oriented Concepts Used

Table 5.2: Where each concept appears in the code

Concept	Where it appears	What it achieves
Abstraction	`Question` declares `validate`, `getCorrectAnswer` and `displayQuestion` without saying how they work.	`Student.answerQuestion` calls `question.validate(letter)` and `Demo` calls `q.displayQuestion();` neither knows the rules of any particular question type.
Inheritance	`MCQ extends Question`.	`MCQ` reuses the fields and constructor of `Question` and adds only what is specific to lettered options.
Polymorphism	Calls made through a `Question` reference are resolved to the `MCQ` methods at run time.	New question types could be added without changing the calling code. At present `MCQ` is the only concrete type (see Chapter 9).

Concept	Where it appears	What it achieves
Encapsulation	Private fields everywhere; `Quiz.getQuestions()` and `getAnswerKey()` return read-only views; `Question` copies the options array and returns clones; a response can change only through `submitAnswer` and `markSubmitted`.	Outside code cannot alter a quiz, its answer key or a submitted response by accident.
Composition	A `Quiz` owns its `Timer` (created in `startQuiz`) and its list of questions.	The timer and the questions have no meaning outside their quiz.
Aggregation and association	`QuizManager` stores quizzes; a `Quiz` holds the responses registered by students; a `Student` holds its own responses and the quizzes joined.	Objects refer to each other without owning each other's life cycle.
Multithreading and synchronisation	A watcher thread in `Demo`; `synchronized` methods in `Student`, `Quiz` and `StudentResponse`; `volatile` flags in `Quiz` and `Timer`; an `AtomicBoolean` in `Demo`.	The auto-submit can run while the main thread waits for input, and only one of them can finish a response.
Exceptions	`IllegalStateException` and `IllegalArgumentException` for refused actions; `IOException` for file problems.	Errors reach the console layer as clear messages instead of crashing the program.

5.5 Use Case Diagram

Figure 5.3 shows what each actor can do. The teacher prepares and manages quizzes and reads the outcome; the student joins, answers, submits and views the score. The timer is a system actor: when it reaches zero it starts the automatic submission. Three use cases (submit quiz, auto-submit on time-up and end quiz and grade) include the same step, auto-grading of a response, which is why that step is drawn once and shared.

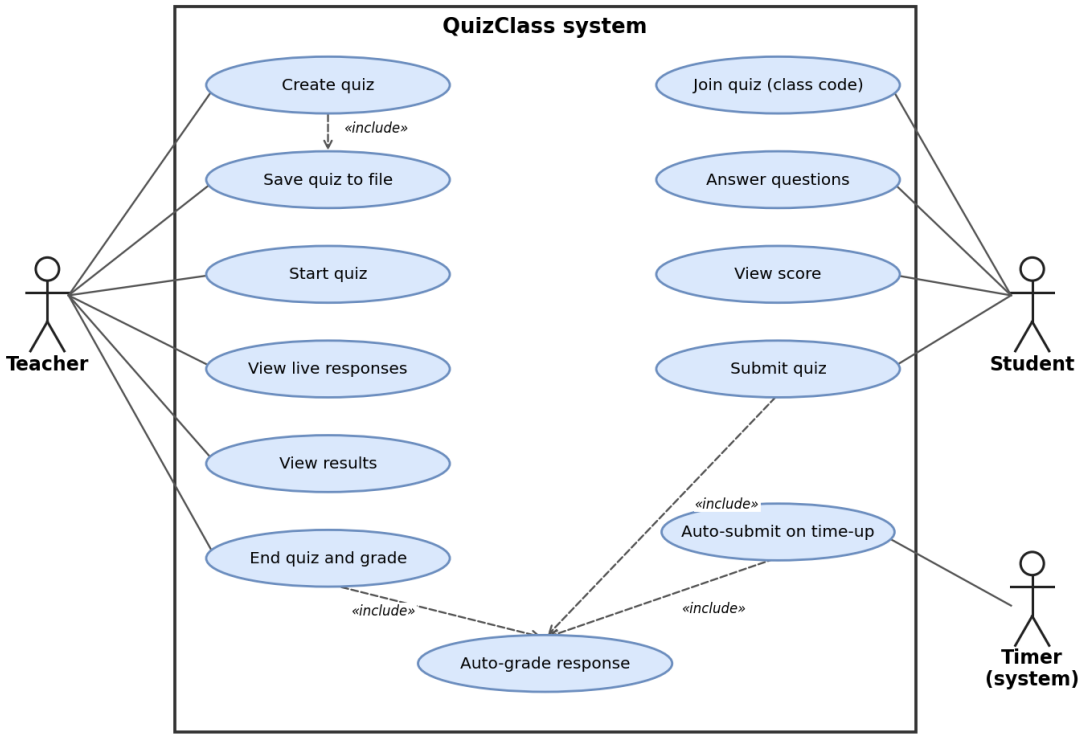


Figure 5.3: Use case diagram

Table 5.3: Main use cases

Use case	Actor	Summary
Create quiz	Teacher	Enters a title, a duration and the questions with four options and the correct letter; the system issues a class code and saves the quiz.
Start quiz	Teacher	Starts the countdown; students can now join and answer.
Join quiz	Student	Enters class code, ID and name; the system finds the quiz and creates (or reloads) the student's response.
Answer questions	Student	Gives one letter per question; invalid letters are refused.
Submit quiz	Student	Finishes the attempt; the response is graded and saved.
Auto-submit on time-up	Timer	When time runs out, the answers given so far are graded and saved.
View score	Student	Sees the percentage; a student who returns later sees the saved score.
View live responses / View results	Teacher	Sees who joined, how many questions each answered, each percentage and the class average.
End quiz and grade	Teacher	Closes the quiz and grades any response not yet submitted.

5.6 Sequence Diagrams

5.6.1 Join, answer and submit

Figure 5.4 follows one student through a normal attempt. `Demo` asks `Student` to join; `Student` asks `QuizManager` for the quiz, creates the response and registers it with the quiz so the teacher

can see it (messages 1–6). For every question, `Demo` displays the question and passes the answer to `Student.answerQuestion`, which checks that the quiz is still open, asks the `Question` to validate the letter and only then stores it in the response (messages 7–13). Finally `submitQuiz` grades the response by comparing it with the answer key and marks it submitted, and `Demo` saves it and prints the percentage (messages 14–21).

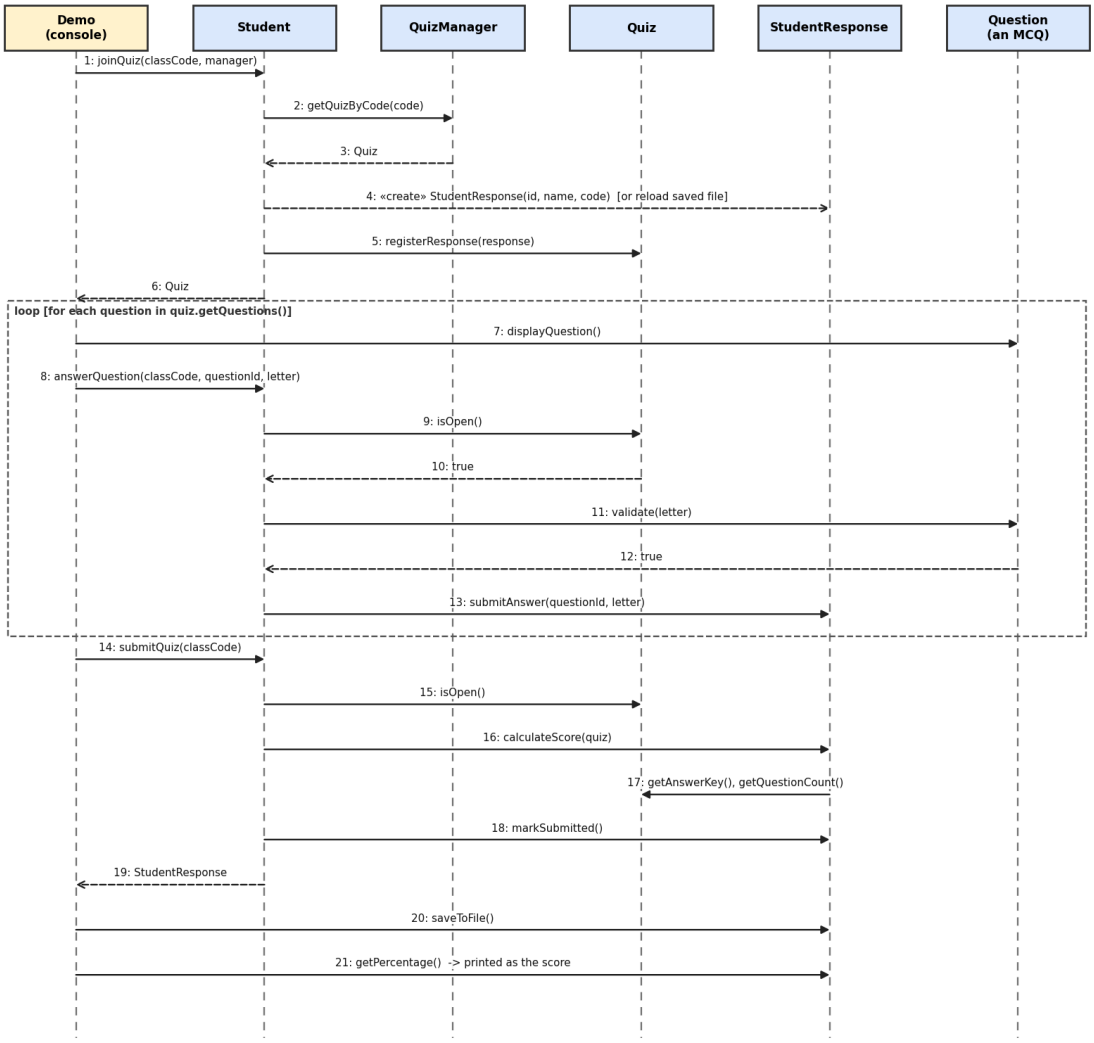


Figure 5.4: Sequence diagram: join, answer and submit

5.6.2 Auto-submit when time is up

Figure 5.5 shows what happens when the student has not finished in time. While the main thread is blocked waiting for the student to type, a **watcher thread** checks `Quiz.isOpen()` every 500 milliseconds. `isOpen()` asks the `Timer` whether time is up. As soon as the quiz is no longer open, the watcher calls `Student.autoSubmit`, which grades what was answered and marks the response submitted. The watcher saves the response, prints the result and sets a flag that the main thread reads after the student presses Enter, so the main thread leaves the question loop and returns to the menu. The timer itself never raises an event; it is a clock that other code polls.

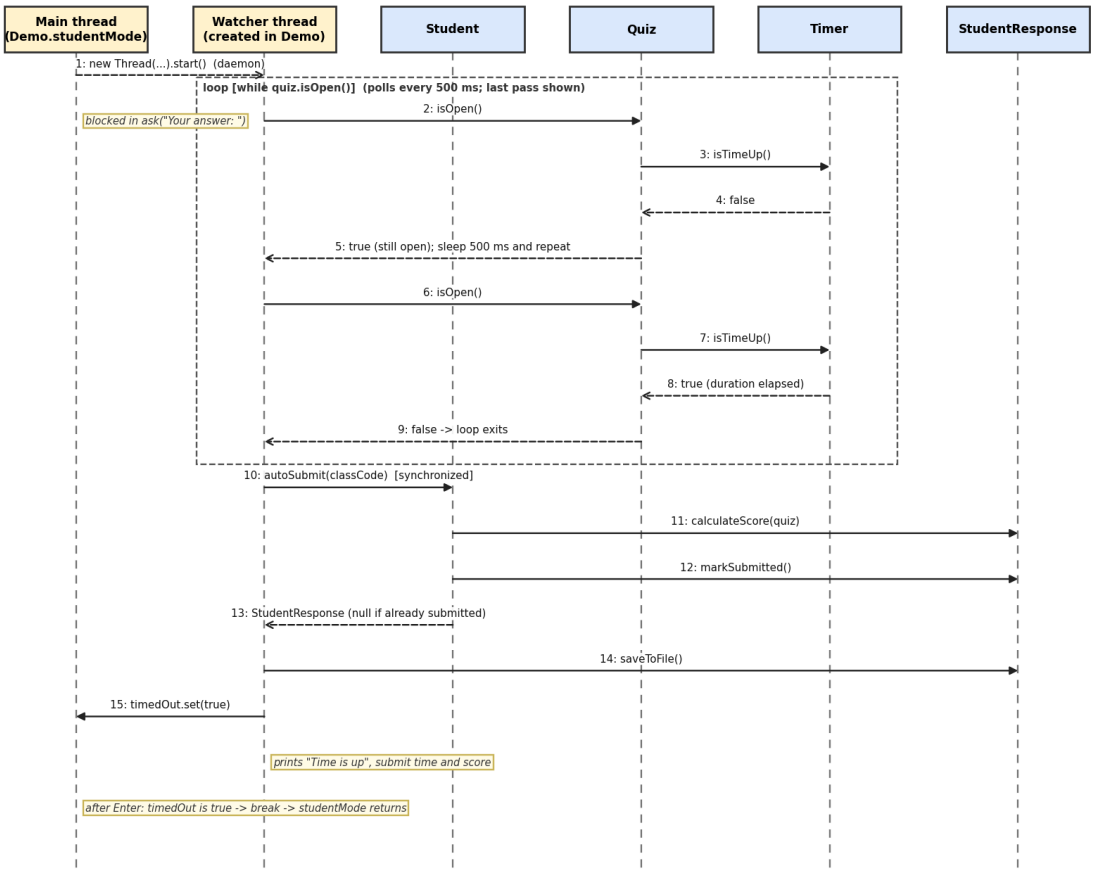
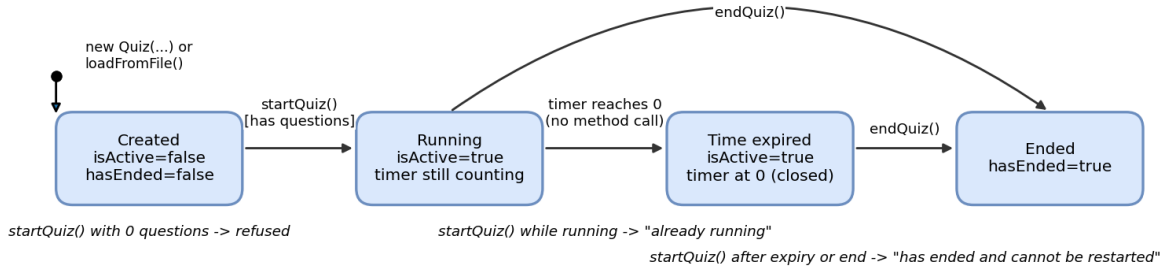


Figure 5.5: Sequence diagram: automatic submission at time-up

5.7 Life Cycle of a Quiz

A quiz moves through a small number of states, shown in Figure 5.6. It is created (or loaded from its file) in the *Created* state. `startQuiz()` moves it to *Running* if it has at least one question. When the timer reaches zero the quiz is closed without any method being called (`isOpen()` becomes false), and the teacher can finish it with `endQuiz()`, which sets the *Ended* state for good. A running quiz can also be ended directly. Once a quiz has expired or ended, `startQuiz()` refuses with the message that it has ended and cannot be restarted.



A quiz loaded from its file after a restart returns to Created: running/ended state is not persisted.

Figure 5.6: Life cycle of a Quiz

5.8 Threading Design

The only concurrency in the system is the auto-submit. Two threads can reach the same student's response at nearly the same moment: the main thread, when the student presses Enter on the last answer, and the watcher thread, when the clock runs out. If both finished the response, it could be graded twice or answered after submission. The design prevents this in three ways.

- `Student.answerQuestion`, `submitQuiz`, `autoSubmit` and `viewScore` are `synchronized`, so they run one at a time for a given student.
- `autoSubmit` returns `null` when the response is already submitted, and `submitQuiz` refuses with “Already submitted.”. Whichever thread arrives second therefore does nothing. The console code handles both outcomes.
- The state flags that the watcher reads (`isActive` and `hasEnded` in `Quiz`, `isActive` in `Timer`) are `volatile`, so the watcher always sees the latest value written by the main thread. `StudentResponse` methods that read or change its state are synchronized as well.

The watcher is a daemon thread, so it never keeps the program alive. When the student submits normally the main thread interrupts the watcher so that it stops polling.

6. Implementation

6.1 Project Structure and How to Run

The project consists of eleven Java source files in one folder, with no packages and no external libraries. `Demo` is the program that is run for the demonstration; `Main` is a small early smoke test of quiz creation, and `StudentApp` is the original student-only console flow that was written to the interface specification before the two halves were merged into `Demo`.

Compile and run

```
javac *.java
java Demo
```

When the program starts it loads every saved quiz from the `quizzes` folder. Responses are written to a `responses` folder, and both folders are created next to where the program is run.

Table 6.1: Source files

File	Role
Question.java, MCQ.java	Abstract question and the multiple-choice implementation
Quiz.java, Timer.java	The quiz (questions, key, state, persistence) and its countdown clock
StudentResponse.java, Student.java	A student's answers and the student's actions
Teacher.java, QuizManager.java	Teacher actions and the registry of quizzes
Demo.java	Console driver with teacher and student menus (the program to run)
StudentApp.java, Main.java	Earlier student-only console flow and a smoke test

6.2 Question and MCQ

`Question` is the abstract base class. It stores the question ID, text, options and correct answer as `protected` fields and leaves three behaviours abstract. The constructor copies the options array, so a caller cannot change a question after creating it by modifying the array it passed in.

Question.java (declarations)

```
public abstract class Question {
    protected int questionId;
    protected String questionText;
    protected String[] options;
    protected String correctAnswer;

    // constructor (copies the options array) and getters omitted

public abstract class Question {
    public abstract boolean validate(String answer);
    public abstract String getCorrectAnswer();
    public abstract void displayQuestion();
}
```

`MCQ` implements those three methods for lettered options. Its constructor rejects a blank text, fewer than two options, and a correct answer that is not one of the option letters, so an invalid

question can never exist. Answers are normalised (trimmed and converted to upper case) before they are compared, so “d”, “ D ” and “D” are the same answer.

MCQ constructor

```
/**
 * @throws IllegalArgumentException if the text is blank, there are fewer than
 *                                  2 options, or the correct answer is not one
 *                                  of the option letters
 */
public MCQ(int questionId, String questionText,
        String[] options, String correctAnswer) {
    super(questionId, questionText, options, normalize(correctAnswer));
    if (questionText == null || questionText.trim().isEmpty()) {
        throw new IllegalArgumentException("Question text cannot be empty.");
    }
    if (options == null || options.length < 2) {
        throw new IllegalArgumentException("A question needs at least 2 options.");
    }
    if (!isValidOption(this.correctAnswer)) {
        throw new IllegalArgumentException(
                "Correct answer must be one of the option letters, got: " + correctAnswer);
    }
}
```

MCQ.isValidOption

```
/** A valid answer is exactly one letter that maps to an existing option index. */
private boolean isValidOption(String answer) {
    String letter = normalize(answer);
    if (letter.length() != 1) {
        return false;
    }
    int index = letter.charAt(0) - 'A';
    return index >= 0 && index < options.length;
}
```

MCQ.normalize

```
/** Trims and upper-cases so "d", " D " and "D" are treated the same; null becomes "". */
private static String normalize(String s) {
    return (s == null) ? "" : s.trim().toUpperCase();
}
```

`validate` simply delegates to `isValidOption`. The index calculation `letter.charAt(0) - 'A'` turns a letter into a position in the options array, so a four-option question accepts exactly A, B, C and D and refuses E or the empty string.

6.3 Timer

`Timer` is a passive clock. `startTimer` records the start time and the duration in milliseconds. `getRemainingTime` computes what is left, never below zero, and returns zero if the timer was stopped. `isTimeUp` is true when nothing remains. The `isActive` flag is `volatile` because the watcher thread reads it while the main thread may write it.

Timer.getRemainingTime

```
/** Milliseconds left; 0 when the time has run out or the timer was stopped. */
public long getRemainingTime() {
    if (!isActive) {
        return 0;
    }
    long elapsed = System.currentTimeMillis() - startTime;
    return Math.max(0, duration - elapsed);
}
```

Timer.isTimeUp

```
public boolean isTimeUp() {
    return getRemainingTime() == 0;
}
```

6.4 Quiz

`Quiz` is the central class. It holds the questions in a list and the answer key in a map from question ID to the correct letter; the key is filled by `addQuestion`, which also rejects a duplicate question ID and refuses any change once the quiz has started.

Table 6.2: Main attributes of Quiz

Attribute	Type	Purpose
classCode	String	The identifier of the quiz; used for lookup and in file names
title, duration	String, int	Title shown to students; duration in minutes
questions	List<Question>	The questions in order
answerKey	Map<Integer, String>	Correct answer for each question ID
isActive, hasEnded	volatile boolean	State flags; volatile because the watcher thread reads them
timer	Timer	Created in `startQuiz`; owned by the quiz
responses	List<StudentResponse>	One response per student who joined; read by the teacher

The life cycle of Figure 5.6 is implemented by three methods. `startQuiz` refuses a quiz that has ended or whose time has run out, one that is already running, and one that has no questions; otherwise it creates the timer and marks the quiz active. `endQuiz` stops the timer and marks the quiz ended. `isOpen` is the single question every other class asks: is the quiz running and is there time left?

Quiz.startQuiz

```
/**
 * Starts the countdown. Refused if the quiz already ended (by endQuiz() or because
 * the time ran out), is already running, or has no questions.
 */
public void startQuiz() {
    if (hasEnded || (timer != null && !isOpen())) {
        throw new IllegalStateException("Quiz " + classCode + " has ended and cannot be
restarted.");
    }
    if (isActive) {
        throw new IllegalStateException("Quiz " + classCode + " is already running.");
    }
    if (questions.isEmpty()) {
        throw new IllegalStateException("Quiz " + classCode + " has no questions and cannot
be started.");
    }
    timer = new Timer();
    timer.startTimer(duration);
    isActive = true;
}
```

Quiz.endQuiz

```
/** Ends the quiz now. Only a running quiz can be ended, and only once. */
public void endQuiz() {
    if (hasEnded) {
        throw new IllegalStateException("Quiz " + classCode + " has already ended.");
    }
    if (!isActive) {
        throw new IllegalStateException("Quiz " + classCode
                + " is not running. It has not been started in this session.");
    }
    if (timer != null) {
        timer.stopTimer();
    }
    isActive = false;
    hasEnded = true;
}
```

Quiz.isOpen

```
/**
 * True while students can answer: the quiz was started, was not ended, and the
 * timer has not run out. Time running out closes the quiz without any method call.
 */
public boolean isOpen() {
    return isActive && timer != null && !timer.isTimeUp();
}
```

Note that time running out closes the quiz through `isOpen()` becoming false; no method is called at that moment. This is why the watcher thread has to poll. `getQuestions()` and `getAnswerKey()` return unmodifiable views, so only `addQuestion` can change them. `registerResponse` and `getResponses` are synchronized because the teacher reads the list while student sessions add to it.

6.5 StudentResponse

A `StudentResponse` stores one student's answers for one quiz: the student ID and name, the class code, a map from question ID to the chosen letter, the percentage and a submitted flag. `submitAnswer` stores or overwrites an answer and is refused once the response is submitted. `markSubmitted` locks the response and stamps the submission time once.

StudentResponse.submitAnswer

```
/**
 * Stores (or overwrites) the answer for a questionId. Not allowed after
 * submission.
 */
public synchronized void submitAnswer(int questionId, String letter) {
    if (submitted) {
        throw new IllegalStateException("Already submitted.");
    }
    if (letter == null || letter.trim().isEmpty()) {
        throw new IllegalArgumentException("Answer cannot be empty.");
    }
    answers.put(questionId, letter.trim().toUpperCase());
}
```

Grading is in `calculateScore`. It walks through the answers the student actually gave and counts those that equal the answer key entry for the same question ID. The percentage is divided by the total number of questions in the quiz, not by the number answered, so an unanswered question is simply a missing match and counts as wrong.

StudentResponse.calculateScore

```
/**
 * Compares each stored answer with the quiz's answer key (same questionId keys)
 * and sets the percentage.
 */
public synchronized void calculateScore(Quiz quiz) {
    Map<Integer, String> key = quiz.getAnswerKey();
    int correct = 0;
    // Unanswered questions have no entry in 'answers', so they simply count as wrong.
    for (Map.Entry<Integer, String> entry : answers.entrySet()) {
        String right = key.get(entry.getKey());
        if (right != null && right.equalsIgnoreCase(entry.getValue())) {
            correct++;
        }
    }
    int total = quiz.getQuestionCount(); // divide by ALL questions, not just the answered
ones
    percentage = (total == 0) ? 0 : (correct * 100) / total;
}
```

6.6 Student

`Student` keeps two maps keyed by class code: the responses and the quizzes joined. `joinQuiz` looks the quiz up through the manager, returns `null` for an unknown code, and otherwise creates the student's response (or reloads a saved one, so that a returning student cannot retake the quiz) and registers it with the quiz.

Student.joinQuiz

```
/**
 * Looks the quiz up by class code and creates (or reloads) this student's response.
 * Returns null if there is no such quiz. It does NOT check whether the quiz is open:
 * the caller (Demo) decides that, so a student can still join a closed quiz to view
 * a saved score.
 */
public Quiz joinQuiz(String classCode, QuizManager manager) {
    if (classCode == null) {
        return null;
    }
    String code = classCode.trim();
    Quiz quiz = manager.getQuizByCode(code);
    if (quiz == null) {
        return null;
    }

    joinedQuizzes.put(code, quiz);
    if (!responses.containsKey(code)) { // joining again keeps the existing response
        StudentResponse r = loadSavedOrNew(code, quiz);
        responses.put(code, r);
        quiz.registerResponse(r); // so the teacher can see this student
    }
    return quiz;
}
```

`answerQuestion` is the gatekeeper for every answer. It checks, in order: the student has joined; the quiz is still open; the question exists (found by question ID, never by position); and the letter is valid for that question type. Only then is the answer stored. Each failure throws an exception with a message that the console shows to the student.

Student.answerQuestion

```
/**
 * Stores the answer letter for a questionId.
 *
 * @throws IllegalStateException    if the student has not joined or the quiz is closed
 * @throws IllegalArgumentException if the question does not exist or the letter is not
 *                                  a valid option (checked by Question.validate)
 */
```

```
public synchronized void answerQuestion(String classCode, int questionId, String letter) {
    StudentResponse response = responses.get(classCode);
    if (response == null) {
        throw new IllegalStateException("You have not joined quiz " + classCode + ".");
    }
    Quiz quiz = joinedQuizzes.get(classCode);
    if (!quiz.isOpen()) {
        throw new IllegalStateException("Quiz " + classCode + " is closed. Time is up.");
    }
    Question question = findQuestion(quiz, questionId);
    if (question == null) {
        throw new IllegalArgumentException("Question " + questionId + " does not exist.");
    }
    if (!question.validate(letter)) {
        throw new IllegalArgumentException("Invalid answer '" + letter
                + "' for question " + questionId + ". Please choose one of the listed
options.");
    }
    response.submitAnswer(questionId, letter);
}
```

`submitQuiz` is the normal submission path and `autoSubmit` is the one used at time-up. They differ in one important way: `submitQuiz` refuses an already submitted response or a closed quiz and tells the caller, whereas `autoSubmit` quietly returns `null` when there is nothing left to submit. Both are synchronized on the student.

Student.submitQuiz

```
/**
 * Grades the response, marks it submitted and returns it.
 *
 * @throws IllegalStateException if not joined, already submitted, or the quiz is closed
 */
public synchronized StudentResponse submitQuiz(String classCode) {
    StudentResponse response = responses.get(classCode);
    if (response == null) {
        throw new IllegalStateException("You have not joined quiz " + classCode + ".");
    }
    if (response.isSubmitted()) {
        throw new IllegalStateException("Already submitted.");
    }
    if (!joinedQuizzes.get(classCode).isOpen()) {
        throw new IllegalStateException("Quiz " + classCode + " is closed. Time is up.");
    }
    response.calculateScore(joinedQuizzes.get(classCode));
    response.markSubmitted();
    return response;
}
```

Student.autoSubmit

```
/**
 * Called by the watcher thread when time runs out. Grades whatever was answered
 * (unanswered questions count as wrong). Returns null if there is nothing to submit
 * because the student already submitted.
 */
public synchronized StudentResponse autoSubmit(String classCode) {
    StudentResponse response = responses.get(classCode);
    if (response == null || response.isSubmitted()) {
        return null;
    }
    response.calculateScore(joinedQuizzes.get(classCode));
    response.markSubmitted();
    return response;
}
```

6.7 Teacher and QuizManager

`Teacher.createQuiz` generates a random four-digit code, draws a new one while the code is already in use, builds the `Quiz` and hands it to the manager. The quiz number is one more than

the highest in use, so numbers stay distinct after quizzes are loaded from files. The teacher keeps no list of its own: `QuizManager` is the single place where quizzes are stored.

Teacher.createQuiz

```
/** Creates a quiz with a fresh class code and registers it with the manager. */
public Quiz createQuiz(String title, int duration) {
    String code = generateClassCode();
    while (manager.getQuizByCode(code) != null) { // never reuse a code
        code = generateClassCode();
    }
    Quiz quiz = new Quiz(nextQuizId(), code, title, duration);
    manager.createAndStoreQuiz(quiz);
    return quiz;
}
```

Teacher.nextQuizId

```
/** One more than the highest id in use, so ids stay unique after quizzes are loaded from
files. */
private int nextQuizId() {
    int next = 1;
    for (Quiz q : manager.getAllQuizzes()) {
        next = Math.max(next, q.getQuizId() + 1);
    }
    return next;
}
```

Teacher.generateClassCode

```
/** A random 4-digit code (1000-9999). Uniqueness is enforced in createQuiz. */
public String generateClassCode() {
    return String.valueOf(1000 + random.nextInt(9000));
}
```

`viewLiveResponses` prints each registered student with the number of questions answered and whether the response is submitted. `viewResults` prints each submitted student's percentage and the class average. `gradeAllResponses` grades, marks and saves every response that is still unsubmitted, and is called when the teacher ends a quiz. `QuizManager` is a thin wrapper around a `HashMap` from class code to `Quiz`; its loading method is described in Chapter 7.

6.8 Demo: the Console Driver

`Demo` contains the menus and the input helpers, and no grading logic. Teacher mode offers create quiz, load a sample quiz, list, start, view live responses, end and grade, and view results. Input helpers (`ask`, `askNonEmpty`, `askInt`, `askPositiveInt`) re-ask until the input is valid, so a blank title, a blank question or a non-numeric duration never reaches the model. If the input stream ends, the program exits cleanly.

Student mode is the longest method. It asks for the class code, ID and name; refuses a quiz that is not open unless this student already has a saved result to view; joins; shows the saved score instead of restarting when one exists; starts the watcher thread; and then loops over the questions. The watcher is the implementation of Figure 5.5:

Demo.studentMode: the watcher thread (output lines abbreviated)

```
Thread watcher = new Thread(() -> {
    try {
```

```
        while (quiz.isOpen()) {
            Thread.sleep(500);
        }
    } catch (InterruptedException e) {
        return; // the student submitted normally, so the watcher is no longer needed
    }
    StudentResponse auto = student.autoSubmit(code); // null if already submitted
    if (auto != null) {
        timedOut.set(true);
        saveResponse(auto);
        // (prints the time-up message, the submission time and the score)
    }
});
watcher.setDaemon(true); // never keeps the program alive on its own
watcher.start();
```

The loop polls `quiz.isOpen()` every 500 ms. When it exits, `student.autoSubmit(code)` grades the response; a non-null result means this thread did the submission, so it saves the response, reports the score and sets the `timedOut` flag. The `AtomicBoolean` is read by the main thread, which is blocked waiting for the student's next line; after the student presses Enter it sees the flag and returns to the menu. If the student submits normally first, the main thread interrupts the watcher, which then simply returns.

If the student answers every question before the deadline, `student.submitQuiz` is called. Should the quiz close at the very last moment, `submitQuiz` refuses and the answers given so far are auto-submitted instead, so there is never a state in which an attempt is lost.

6.9 StudentApp and Main

`StudentApp` implements the original user-interaction specification: it builds a teacher and a one-minute quiz in memory, starts it, then asks for the class code, ID and name, calls `joinQuiz`, displays each question, passes each answer to `answerQuestion` (asking again if it throws), calls `submitQuiz` and prints the percentage. It was used to check the student classes before they were merged into `Demo`, and it contains no grading logic. `Main` creates a quiz and one question and prints the answer key and two calls of `MCQ.isCorrect`; it was the first smoke test of the quiz-side classes.

7. Persistence and File Formats

7.1 Overview

Quizzes and responses are stored as plain text files so that they survive a restart and can be opened in any text editor, which also made them easy to inspect while testing. There is one file per quiz and one file per student per quiz.

Table 7.1: Files written by the system

Folder and file name	Written by	Contents
quizzes/quiz_<classCode>.txt	`Quiz.saveToFile`, when a quiz is created	Quiz metadata and all questions with their correct letters
responses/response_<classCode>_<studentId>.txt	`StudentResponse.saveToFile`, when a response is submitted	Student ID and name, the answer given for each question, and the submission time

7.2 The Quiz File

The quiz file has a metadata line and one line per question. Fields are separated by the pipe character. Figure 8.2 in Chapter 8 shows a real file produced by the system.

Format of quizzes/quiz_<classCode>.txt

```
QUIZ_METADATA
QuizId|<number>|Title|<title>|Duration|<minutes>|Code|<classCode>
QUESTIONS
<questionId>|<text>|<correct letter>|<option A>|<option B>|<option C>|<option D>
```

7.3 The Response File

The response file starts with the student ID and name, then has one line for every question that was answered, in question order, and ends with the submission time. Questions that were left unanswered have no line. Figure 8.7 shows a real response file.

Example of responses/response_5913_1.txt

```
StudentId|1
StudentName|ajay
Question|1|Answer|D
Question|2|Answer|B
Question|3|Answer|A
SubmittedAt|24 Oct 2026, 19:27:54 IST
```

The percentage is deliberately **not** stored. It is recomputed from the saved answers and the quiz's answer key whenever a response is loaded, so there is a single source of truth for grading.

The consequence, noted in Chapter 9, is that changing a quiz file afterwards would change old scores.

7.4 Loading at Start-up

`Demo.main` calls `QuizManager.loadQuizzesFromFile()`. For every file that matches `quiz_*.txt` in the quizzes folder the manager asks `Quiz.loadFromFile` to rebuild the quiz, then asks the quiz to load the responses saved for it with `loadSavedResponses`, which reads every `response_<classCode>_*.txt` file, recalculates each score and registers the response so that the teacher sees the results after a restart.

Quiz.loadFromFile

```
/**
 * Reads a quiz saved by saveToFile. Throws IOException if the file is damaged
 * (missing metadata, a short or malformed line, a duplicate question id...).
 * Only multiple-choice questions can be stored, so every question line becomes an MCQ.
 */
public static Quiz loadFromFile(Path file) throws IOException {
    List<String> lines = Files.readAllLines(file, StandardCharsets.UTF_8);
    Quiz quiz = null;
    boolean inQuestions = false; // false while reading the metadata line, true after
"QUESTIONS"
    for (String line : lines) {
        if (line.trim().isEmpty() || line.equals("QUIZ_METADATA")) {
            continue;
        }
        if (line.equals("QUESTIONS")) {
            inQuestions = true;
            continue;
        }
        try {
            String[] fields = line.split("\\|", -1); // -1 keeps trailing empty fields
            if (!inQuestions) {
                // QuizId|<id>|Title|<title>|Duration|<minutes>|Code|<code>
                quiz = new Quiz(Integer.parseInt(fields[1]), fields[7], fields[3],
                        Integer.parseInt(fields[5]));
            } else {
                if (quiz == null) {
                    throw new IOException("Questions before metadata in " + file);
                }
                // <id>|<text>|<correct letter>|<option A>|<option B>|...
                String[] options = Arrays.copyOfRange(fields, 3, fields.length);
                quiz.addQuestion(new MCQ(Integer.parseInt(fields[0]), fields[1], options,
fields[2]));
            }
        } catch (ArrayIndexOutOfBoundsException | IllegalArgumentException e) {
            // NumberFormatException is an IllegalArgumentException, so bad numbers land
here too
            throw new IOException("Bad line in quiz file " + file + ": " + line);
        }
    }
    if (quiz == null) {
        throw new IOException("No quiz metadata in " + file);
    }
    return quiz;
}
```

The parser splits each line at the pipe character. The first line after `QUIZ_METADATA` creates the quiz; each line after `QUESTIONS` becomes an `MCQ` whose options are all the fields after the correct letter. Because the file stores only multiple-choice questions, the loader creates `MCQ` objects directly.

7.5 Duplicate Prevention Across Restarts

When a student joins, `Student.loadSavedOrNew` checks whether a response file exists for that class code and student ID. If it does, the response is loaded, graded and marked submitted, so the student sees the saved score and cannot start the quiz again, even after closing and reopening the program. The console driver performs the same check before refusing a closed quiz, which is what lets a student view an old score after the quiz has ended.

7.6 Error Handling

- **Damaged files:** a quiz or response file with a missing field, a bad number, a duplicate question ID or an invalid correct letter makes the loader throw an `IOException` that names the file and line. The manager prints a warning and skips that file; the rest load normally.
- **The separator character:** a pipe inside a title, question or option would corrupt the line, so it is replaced by a space (as are line breaks) when the file is written.
- **File names:** a class code is accepted for a file name only if it consists of letters, digits, hyphens or underscores, so a crafted code cannot point outside the data folders.
- **Write failures:** if a file cannot be written the program prints a warning and carries on; the quiz or response stays in memory for the session.

7.7 What Is Not Persisted

Whether a quiz is running or has ended, and how much time is left, are not saved. A quiz loaded from its file is always in the *Created* state and must be started again. This keeps the file format small, but it has consequences that are listed in Chapter 9.

8. Testing and Results

8.1 Test Approach

The system was tested from the console against the final sources, compiled with `javac *.java` and started with `java Demo`. Two kinds of test were used. First, a complete session was run by hand and recorded in screenshots: a teacher creates, saves and starts a quiz, two students take it (one finishes, one runs out of time), and the teacher reviews the responses and results. These screenshots are Figures 8.1 to 8.12. Second, error cases and restarts were repeated with scripted console input, so that every run used exactly the same keystrokes; one of these runs waited for a one-minute quiz to expire in real time.

Each test case below names the scenario, the expected behaviour and the evidence. Where the evidence is a figure, the screenshot is included in Section 8.3.

8.2 Test Cases

Table 8.1: Test cases and results

ID	Scenario	Expected result	Evidence	Result
TC-01	Teacher creates a quiz with title, duration and three questions.	Quiz is created, a four-digit class code is shown, the quiz is saved.	Fig. 8.1	Pass
TC-02	Saved quiz file is written and listed.	File quizzes/quiz_<code>.txt contains metadata and one line per question; the quiz appears in the list with code, title, duration and question count.	Fig. 8.2, 8.3	Pass
TC-03	Teacher starts the quiz.	Confirmation that students can join; countdown begins.	Fig. 8.4	Pass
TC-04	Student joins with a valid class code, ID and name.	Quiz title and the first question are shown with the time left.	Fig. 8.5	Pass
TC-05	Student answers every question correctly (lower-case letters).	Answers accepted; manual submission; score 100%.	Fig. 8.6	Pass
TC-06	Submitted response is saved.	responses/response_<code>_<id>.txt lists each answer and the submission time.	Fig. 8.7	Pass
TC-07	Same student enters the quiz again.	Second attempt is refused and the saved score is shown.	Fig. 8.8	Pass
TC-08	A student leaves the last question unanswered until time runs out.	Quiz is auto-submitted; two correct answers out of three give 66%.	Fig. 8.9	Pass
TC-09	Teacher views live responses.	Each student, answered/total and submitted status are listed.	Fig. 8.10	Pass
TC-10	Teacher views results.	Each percentage and the class average (100 and 66 give 83%).	Fig. 8.11	Pass
TC-11	Teacher ends the quiz and grades.	Quiz ended; all responses graded.	Fig. 8.12	Pass
TC-12	Student enters an unknown class code.	“Invalid class code”; no further questions are asked.	Scripted run	Pass
TC-13	Student types an invalid letter (for example Z).	Message shown and the same question is asked again.	Scripted run	Pass

ID	Scenario	Expected result	Evidence	Result
TC-14	Student answers every question wrongly.	Score 0%.	Scripted run	Pass
TC-15	A new student tries to join after the quiz has closed.	Refused: the quiz is not open.	Scripted run	Pass
TC-16	Teacher tries to start a quiz that has ended or timed out.	Refused: the quiz has ended and cannot be restarted.	Scripted run	Pass
TC-17	Program is restarted after quizzes and responses were saved.	Quizzes and responses are loaded; the teacher sees the results; a student who already submitted sees the saved score.	Scripted run	Pass
TC-18	A damaged quiz file is present in the quizzes folder.	A warning names the file and start-up continues with the other quizzes.	Scripted run	Pass
TC-19	Teacher starts a quiz that has no questions.	Refused: the quiz has no questions.	Scripted run	Pass
TC-20	Blank title, blank question text or blank option is entered while creating a quiz.	The program asks again instead of failing.	Scripted run	Pass

In the scripted runs a three-question quiz was taken by three students: the all-correct student scored 100%, the all-wrong student 0% and a student with two correct answers 66%. In the real-time expiry run a student answered only the first question of a one-minute quiz and then waited; after the minute had passed the quiz was auto-submitted with 33%, and starting the quiz again was refused.

8.3 Walk-through of a Complete Session

8.3.1 The teacher creates, saves and starts a quiz

In teacher mode the teacher chooses *Create quiz* and enters the title “Java OOP Basics”, a duration of five minutes and three questions, each with four options and the correct letter. The system issues the class code **5913** and saves the quiz (Figure 8.1).

```
< QUIZCLASS >
1. Teacher mode
2. Student mode
3. Exit
Choice: 1

* TEACHER *
1. Create quiz
2. Load sample quiz (3 questions)
3. List quizzes
4. Start quiz
5. View live responses
6. End quiz and grade
7. View results
8. Back
Choice: 1
Quiz title: Java OOP Basics
Duration (minutes): 5
Number of questions: 3
Q1 text: Which keyword creates a new object in Java?
  Option A: class
  Option B: this
  Option C: static
  Option D: new
  Correct letter (A-D): d
Q2 text: Which OOP concept lets a class reuse the fields and methods of another class?
  Option A: Encapsulation
  Option B: Inheritance
  Option C: Polymorphism
  Option D: Abstraction
  Correct letter (A-D): b
Q3 text: Which concept lets one method call behave differently for different object types?
  Option A: Polymorphism
  Option B: Encapsulation
  Option C: Compilation
  Option D: Serialization
  Correct letter (A-D): a
Quiz created and saved. Class code: 5913
```

Figure 8.1: Teacher creates a quiz of three questions and receives class code 5913

The saved file is plain text. The first line after `QUIZ_METADATA` carries the quiz number, title, duration and class code; each line after `QUESTIONS` holds the question number, the text, the correct letter and the four options (Figure 8.2). *List quizzes* shows the new quiz together with an older one, quiz 5001, which had been saved in an earlier run and was loaded when the program started (Figure 8.3).

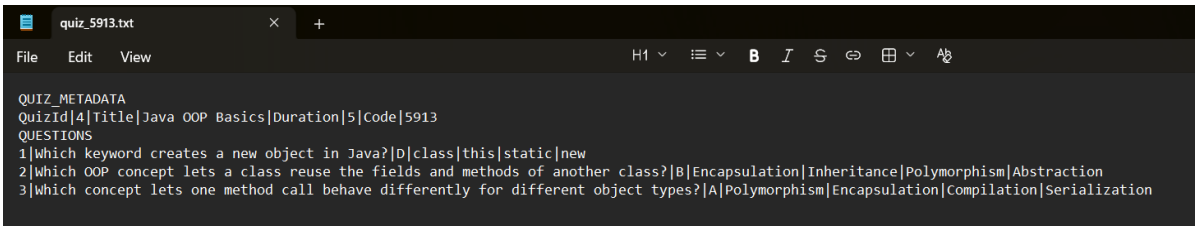


Figure 8.2: The saved file quizzes/quiz_5913.txt

```
* TEACHER *
1. Create quiz
2. Load sample quiz (3 questions)
3. List quizzes
4. Start quiz
5. View live responses
6. End quiz and grade
7. View results
8. Back
Choice: 3
5001 - Java Basics (2 min, 3 questions)
5913 - Java OOP Basics (5 min, 3 questions)
```

Figure 8.3: Listing the saved quizzes (code, title, duration, number of questions)

The teacher then starts the quiz. The system confirms that students can join and that the quiz lasts five minutes (Figure 8.4). From this moment the timer is running.

```
* TEACHER *
1. Create quiz
2. Load sample quiz (3 questions)
3. List quizzes
4. Start quiz
5. View live responses
6. End quiz and grade
7. View results
8. Back
Choice: 4
Class code: 5913
Quiz started. Students can join now (5 minute(s)).
```

Figure 8.4: Teacher starts quiz 5913

8.3.2 A student joins, answers and submits

In student mode the student enters the class code, a student ID and a name. The system shows the quiz title, the time left and the first question with its lettered options (Figure 8.5). The time left is shown before every question.

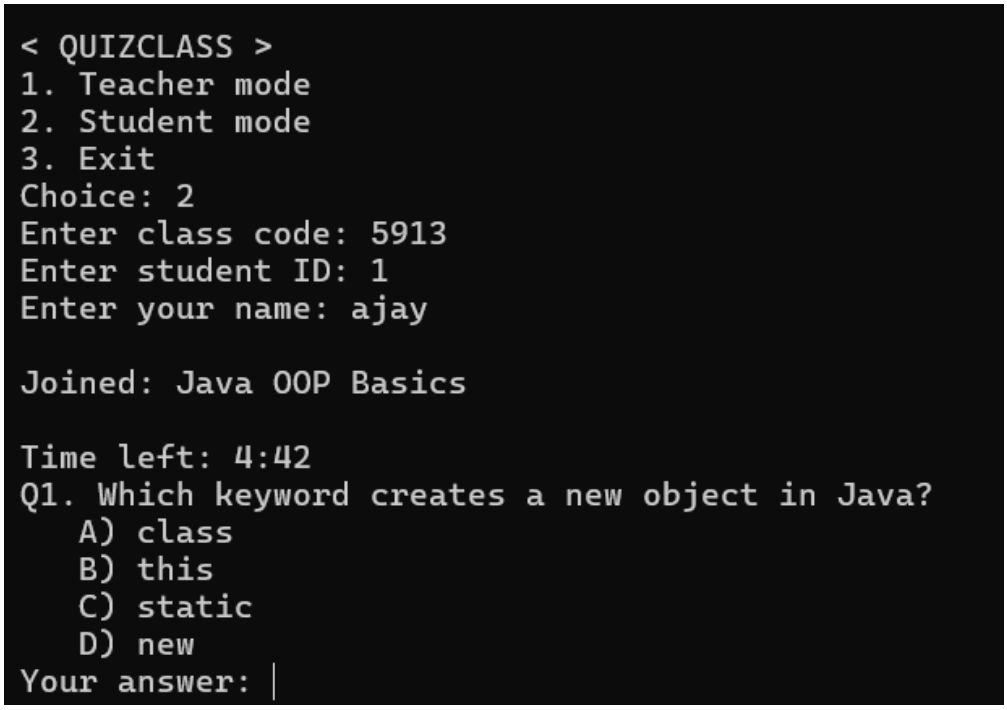


Figure 8.5: Student joins quiz 5913 and sees the first question

The student answers all three questions. The letters were typed in lower case and were accepted, because answers are normalised. After the last answer the quiz is submitted, the submission time is shown and the score is 100% (Figure 8.6).

```
< QUIZCLASS >
1. Teacher mode
2. Student mode
3. Exit
Choice: 2
Enter class code: 5913
Enter student ID: 1
Enter your name: ajay

Joined: Java OOP Basics

Time left: 4:42
Q1. Which keyword creates a new object in Java?
   A) class
   B) this
   C) static
   D) new
Your answer: d

Time left: 3:47
Q2. Which OOP concept lets a class reuse the fields and methods of another class?
   A) Encapsulation
   B) Inheritance
   C) Polymorphism
   D) Abstraction
Your answer: b

Time left: 3:45
Q3. Which concept lets one method call behave differently for different object types?
   A) Polymorphism
   B) Encapsulation
   C) Compilation
   D) Serialization
Your answer: a

Quiz submitted.
Submitted at: 24 Oct 2026, 19:27:54 IST
Your score: 100%
```

Figure 8.6: Student answers all questions, submits and sees a score of 100%

The submitted response has been written to its own file, which lists the student, each answer by question number and the submission time (Figure 8.7).

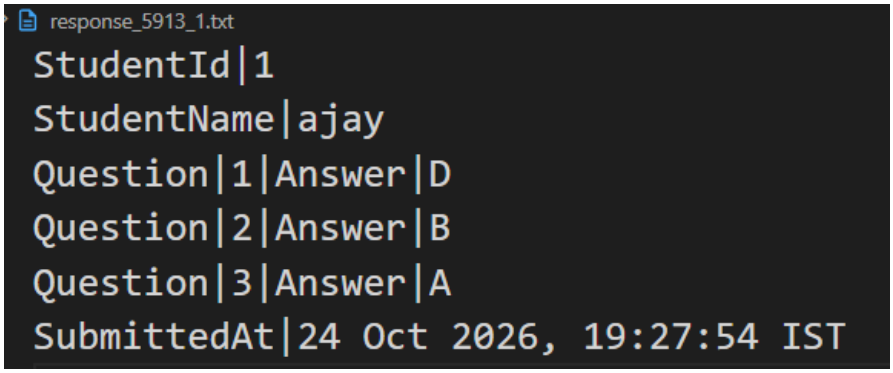


Figure 8.7: The saved response file responses/response_5913_1.txt

When the same student enters the quiz again, the system finds the saved response and refuses a second attempt. It shows the earlier score instead (Figure 8.8). This works even after the program has been restarted, because the check uses the saved file.

```
< QUIZCLASS >
1. Teacher mode
2. Student mode
3. Exit
Choice: 2
Enter class code: 5913
Enter student ID: 1
Enter your name: ajay
You have already taken this quiz. Your score: 100%
```

Figure 8.8: A second attempt is refused and the saved score is shown

8.3.3 Automatic submission at time-up

A second student (ID 2) joins the same quiz with 2 minutes 33 seconds left. The student answers the first two questions correctly but is still reading the third when the clock reaches zero. The watcher thread submits the answers given so far without any action by the student. The screen shows that the time is up, the submission time and the score of 66%, which is two correct answers out of three; the unanswered question counts as wrong (Figure 8.9).

```
< QUIZCLASS >
1. Teacher mode
2. Student mode
3. Exit
Choice: 2
Enter class code: 5913
Enter student ID: 2
Enter your name: aayush

Joined: Java OOP Basics

Time left: 2:33
Q1. Which keyword creates a new object in Java?
   A) class
   B) this
   C) static
   D) new
Your answer: d

Time left: 2:29
Q2. Which OOP concept lets a class reuse the fields and methods of another class?
   A) Encapsulation
   B) Inheritance
   C) Polymorphism
   D) Abstraction
Your answer: b

Time left: 2:26
Q3. Which concept lets one method call behave differently for different object types?
   A) Polymorphism
   B) Encapsulation
   C) Compilation
   D) Serialization
Your answer:
Time is up. Your quiz was auto-submitted.
Submitted at: 24 Oct 2026, 19:31:34 IST
Your score: 66%
```

Figure 8.9: The quiz is auto-submitted when time runs out; two of three answered correctly gives 66%

8.3.4 The teacher reviews the responses

Back in teacher mode, *View live responses* lists each student who joined, how many questions each answered and whether the response is submitted. The quiz is no longer open, and both

responses are submitted: the first student answered 3 of 3 and the second 2 of 3 (Figure 8.10). In this single-console program the teacher menu can only be reached between student sessions, so every response is already submitted by the time the teacher looks; this is discussed as a limitation in Chapter 9.

```
* TEACHER *
1. Create quiz
2. Load sample quiz (3 questions)
3. List quizzes
4. Start quiz
5. View live responses
6. End quiz and grade
7. View results
8. Back
Choice: 5
Class code: 5913
Quiz open: false
ajay (ID 1): 3/3 answered - submitted
aayush (ID 2): 2/3 answered - submitted
```

Figure 8.10: Live responses for quiz 5913

View results prints each student's percentage and the class average. The two scores of 100% and 66% give an average of 83% (Figure 8.11). Finally, *End quiz and grade* closes the quiz and grades any response that is still unsubmitted (Figure 8.12).

```
* TEACHER *
1. Create quiz
2. Load sample quiz (3 questions)
3. List quizzes
4. Start quiz
5. View live responses
6. End quiz and grade
7. View results
8. Back
Choice: 7
Class code: 5913
ajay (ID 1): 100%
aayush (ID 2): 66%
Class average: 83%
```

Figure 8.11: Results and class average

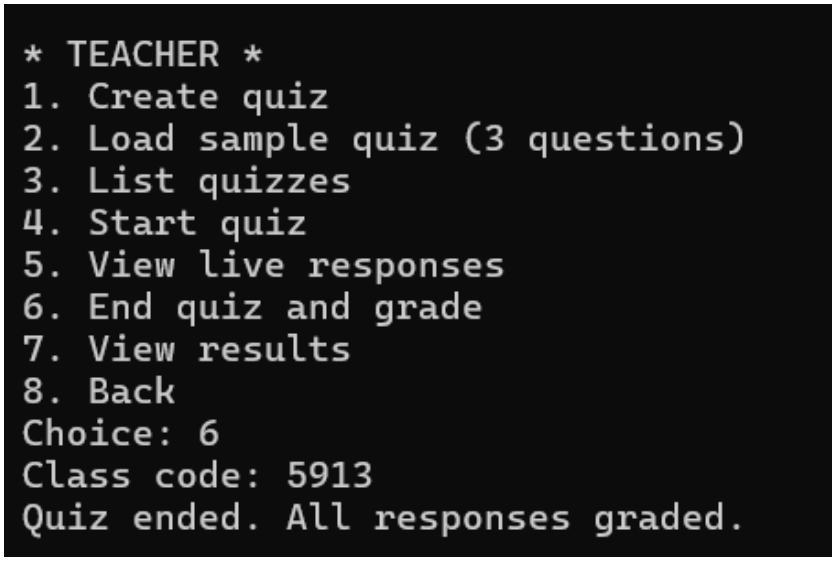


Figure 8.12: Teacher ends the quiz and grades all responses

8.4 Summary of Results

All twenty test cases behaved as expected. Every requirement in Table 2.1 is covered by at least one test case: FR-1 to FR-3 by TC-01, TC-02, TC-17 and TC-20; FR-4 by TC-03, TC-16 and TC-19; FR-5 by TC-04 and TC-12; FR-6 by TC-05 and TC-13; FR-7 by TC-08 and TC-15; FR-8 by TC-05, TC-08 and TC-14; FR-9 by TC-07 and TC-17; FR-10 by TC-09 and TC-10; FR-11 by TC-11; and FR-12 by TC-06 and TC-17.

The tests also showed where the first versions were weak, and these were corrected before the final run: a blank question text once stopped the program with an unhandled exception and is now asked again; a quiz with no questions could be started and is now refused; a closed quiz refused a student who only wanted to see an earlier score, which the console driver now allows; and the message shown for a quiz whose time had simply run out said that it was “already running” instead of “has ended”.

9. Limitations and Future Work

9.1 Known Limitations

The limitations below are known and were accepted to keep the project within its scope. They are listed so that the behaviour of the system is not misrepresented.

Table 9.1: Known limitations

Limitation	Effect	Possible remedy
Console only, no live ticking clock.	The student sees the time left only when a question is displayed, not as a running countdown.	A graphical or web front end with a continuously updated timer.
Teacher and students share one program on one machine; two terminals cannot share a running quiz.	The teacher cannot watch a student mid-quiz: the teacher menu is reachable only between student sessions, so live responses always show finished attempts.	Separate teacher and student programs communicating over sockets or HTTP.
Whether a quiz is running or ended is not saved.	After a restart a loaded quiz is in the Created state, so a finished quiz can be started again. Students who already submitted stay blocked, but students who had not submitted would get a new attempt.	Save the state and the start time with the quiz, or save the end time.
No identity check.	A student is identified only by the ID typed, so entering another student's ID before that student has attempted the quiz takes their attempt, and entering it afterwards shows their score.	Accounts or passwords.
Scores are recomputed, not stored.	Editing a quiz file after the fact would change old scores.	Store the score and a checksum, or make quiz files read-only after use.
Only one question type; the file format has no type field.	Polymorphism is shown through the abstract `Question`, but `MCQ` is the only subclass, and the loader creates `MCQ` objects directly.	Add a type tag to the file and further subclasses, such as true/false or short answer.
Auto-submit is polled every 500 ms.	The submission can lag the deadline by up to half a second.	A scheduled task (`ScheduledExecutorService`) that fires exactly at the deadline.
Class codes are four random digits.	There are only 9000 codes. Uniqueness is checked against the loaded quizzes only, so a leftover response file for a deleted quiz could be matched by a new quiz with the same code.	Longer codes, and checking the response folder as well.
The `|` character is replaced by a space in saved text.	A question that contains a pipe is stored slightly changed.	Escape the separator instead of replacing it.
No networking, analytics, database or mobile application.	These are outside the agreed scope (Section 2.4).	See Section 9.2.

9.2 Future Work

- **Separate teacher and student programs** connected by sockets or a small HTTP server, so students join from their own computers and the teacher can watch responses arrive in real time.
- **More question types** (true/false, multiple answer, short text) with a type tag in the file format, so that polymorphism is exercised by several subclasses.

- **Persisting the quiz state** (started, ended, end time) so that a restart cannot reopen a finished quiz.
- **Accounts and basic authentication** for teachers and students.
- **An exact timer** based on a scheduled task instead of polling, with a visible countdown.
- **Automated tests** with a unit-test framework for grading, validation, persistence and the life-cycle rules.
- **A graphical interface** and reports such as per-question statistics and CSV export.

10. Conclusion

QuizClass meets the objectives set in Chapter 1. A teacher can create a quiz and obtain a four-digit class code; students join with only that code; the quiz runs under a timer that is enforced both when answering and at the deadline; the answers are graded instantly; and the teacher can read the results. Quizzes and responses persist across restarts, and a student cannot take the same quiz twice. The testing in Chapter 8 shows each of these behaviours working, with screenshots of a complete session.

The project also served its purpose as an exercise in object-oriented design. The abstract `Question` and its subclass `MCQ` show abstraction and inheritance; `Quiz` composes a `Timer` and its questions; private state, read-only views and defensive copies show encapsulation; and the auto-submit watcher shows how synchronised methods and volatile flags keep two threads from finishing the same response. We were also helped by fixing the rules in advance (matching quizzes by class code and answers by question ID), which removed a whole class of integration errors between the two halves of the team.

The main lessons were practical. A class diagram drawn in the first week needed to be corrected after coding, and we had to resist adding features: the working timer, the persistence and the tests were worth more than a larger feature list. The limitations in Chapter 9 are honest boundaries rather than hidden defects, and each has a clear remedy. With separate programs, more question types and a persisted quiz state, QuizClass could grow from a laboratory project into a usable classroom tool.

References

[1] Moodle Docs, “Quiz settings” (time limit and the “When time expires” options), documentation for Moodle 3.x. https://docs.moodle.org/36/en/Quiz_settings (viewed 24 October 2026).

[2] Royal Roads University, “Moodle – Quiz activity (staff/faculty)”. https://royalroads.atlassian.net/wiki/x/_A9Z (viewed 24 October 2026).

[3] TU Wien, “Mentimeter” (digital tool catalogue). https://tuwien.at/en/studies/teaching-at-tu-wien/digitally-supported-teaching/digital-tool-catalogue/mentimeter (viewed 24 October 2026).

[4] London School of Economics, “Mentimeter” guide for staff. https://lse.atlassian.net/wiki/x/HgAwlw (viewed 24 October 2026).

[5] Digital Citizen, “Kahoot Game Pin: What It Is And How It Works”. https://www.digitalcitizen.life/what-is-the-kahoot-game-pin/ (viewed 24 October 2026). A third-party guide, not vendor documentation.

[6] Qualtir, “How to add a timer to Google Forms”. https://qualtir.com/blog/how-to-add-timer-to-google-forms (viewed 24 October 2026). A third-party guide, not vendor documentation.

[7] Oracle, Java Platform, Standard Edition API Specification (java.util, java.nio.file, java.util.concurrent). https://docs.oracle.com/en/java/javase/21/docs/api/